

Exercise 1: Breaking the CORS Lock (Policy and Environments)

Context: Security restrictions implemented inside modern web browsers block cross-origin requests between separate local host ports (4200 and 5001) by default. To safely bridge this boundary, you must establish an explicit Cross-Origin Resource Sharing (CORS) policy inside your .NET Core API container while restructuring your Angular service layers to pull URL configurations from environment-agnostic setup parameters.

Part A: Diagnose the Lock in DevTools

Before changing a single line of code, let's observe the browser's security enforcement firsthand:

1. Open your `TmsApi` project and comment out `app.UseCors("AllowAngular");` registered during previous configuration sessions.
2. Launch your .NET Web API project inside one terminal window:

```
dotnet run
```

3. Launch your Angular application inside a second terminal window:

```
ng serve
```

4. Open your browser to `http://localhost:4200` and pull up your Developer Tools (F12).
5. Navigate straight to the **Network** tab and clear the active history log.
6. Trigger an HTTP request action from Angular to your backend API.
7. *Observation State:* Notice that the Network request appears red or fails to complete, and the Console displays the CORS blockage message. Under the hood, the server successfully processed the query and returned a response payload, but Chrome intercepted the payload structure before your TypeScript runtime code could capture it.

[!NOTE]

Part B: Configure a Named CORS Policy in .NET 10

To tell Chrome that our Angular frontend is a trusted partner, we must declare a dedicated CORS policy on the .NET server.

1. Open `appsettings.Development.json` in your Web API project. Add the allowed origin list so we do not hardcode URLs in C# source code:

```
{
  "AllowedOrigins": [
    "http://localhost:4200"
  ]
}
```

2. Open `Program.cs` in your Web API project.
3. Locate the service registration section (before `builder.Build()`) and define a named CORS policy called `"TmsClient"`:

```
// Load allowed origins from appsettings.Development.json
var allowedOrigins =
builder.Configuration.GetSection("AllowedOrigins").Get<string[
]>()
    ?? ["http://localhost:4200"];

// Register the CORS policy in the Dependency Injection
container
builder.Services.AddCors(options =>
{
    options.AddPolicy("TmsClient", policy =>
    {
        policy.WithOrigins(allowedOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials() // Vital for HttpOnly auth
cookies in Session 2
            .SetPreflightMaxAge(TimeSpan.FromMinutes(10));
    });
});
```

4. Scroll down to the HTTP request pipeline configuration section (after `builder.Build()`). Enable the policy using `app.UseCors()`:

```
// CRITICAL: Middleware order matters!
// UseRouting -> UseCors -> UseAuthentication ->
UseAuthorization
app.UseCors("TmsClient");
```

[!CAUTION]

Critical Security Trap to Avoid: Never combine `.AllowAnyOrigin()` with `.AllowCredentials()`. If you attempt to do so, ASP.NET Core will throw an `InvalidOperationException` at server startup. The browser specification strictly forbids wildcard origins when sending authenticated credentials like cookies or auth headers, because doing so would allow any malicious site on the web to make credentialed calls against your user's session.

5. Save `Program.cs`.
6. Stop your API terminal (`Ctrl+C`) and restart it (`dotnet run`). Configuration changes in `Program.cs` require a full process restart to take effect.

[!NOTE]

Part C: Clean Environment Configurations and Domain Models

Hardcoding URLs like `http://localhost:5000` inside your services makes deploying to production painful. Let's create Angular environment configurations to handle API base routes cleanly.

1. Angular 22 does not generate environment files by default. Run the generator in your Angular project terminal:

```
ng generate environments
```

Note: This command creates `src/environments/environment.ts` and `src/environments/environment.development.ts`, while updating `angular.json` automatically.

2. Open `src/environments/environment.development.ts` and set your local API endpoint:

```
export const environment = {  
  production: false,  
  apiUrl: '/api/v1'  
};
```

3. Open `src/environments/environment.ts` (used for production builds):

```
export const environment = {  
  production: true,  
  apiUrl: '/api/v1'  
};
```

4. Verify your Course interface and `PagedResponse<T>` wrapper in `src/app/models/course.model.ts`:

```
public interface Course {  
  id: number;  
  code: string;  
  title: string;  
  maxCapacity: number;  
  enrollmentCount: number;  
  status?: string;  
}  
  
public interface PagedResponse<T> {  
  items: T[];
```

```

totalCount: number;
page: number;
pageSize: number;
totalPages: number;
hasPrevious: boolean;
hasNext: boolean;
}

```

5. Update your CourseService (`src/app/services/course.service.ts`) to use the environment config and modern Angular `@Service()` injection patterns:

```

import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { map } from 'rxjs/operators';
import { environment } from '../../environments/environment';
import { Course, PagedResponse } from
'../models/course.model';

@Service()
public class CourseService {
  private http = inject(HttpClient);
  private readonly base = `${environment.apiUrl}/courses`;

  getAll() {
    return this.http
      .get<PagedResponse<Course>>(this.base, {
        params: { page: '1', pageSize: '50' }
      })
      .pipe(map(response => response.items));
  }
}

```

Part D: Verify the End-to-End Environment Pipeline

Follow these execution steps in order to confirm your environment-aware CORS configurations and service layer mappings are running correctly:

1. **Relaunch the Services:** Ensure your terminal environments are clear, then restart both server instances to pick up your latest configuration and proxy changes:

```

# Terminal 1 – .NET Web API
dotnet run

# Terminal 2 – Angular Frontend Client
ng serve

```

2. **Execute Browser-Level Inspection:** Open `http://localhost:4200` inside your browser and hit **F12** to enter the Developer Tools dashboard.
3. **Trace the Network Headers:** Trigger a manual refresh on the dashboard layout and select the outbound `/api/v1/courses` call inside your **Network** history log:
 - Look for the **Response Headers** section.
 - Verify that the following security parameters populate your headers exactly:

```
Access-Control-Allow-Origin: http://localhost:4200  
Access-Control-Allow-Credentials: true
```

4. **Confirm Application Data Hydration:** Ensure that your course list table renders with live backend rows—confirming that Chrome is no longer dropping response frames and your environmental endpoints are mapping cleanly.